



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 11

Medición de Temperatura y Control de Ventilador con Monitoreo Gráfico

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 20 DE MAYO DE 2026

Sistema de Medición de Temperatura con Sensor LM35,

Control de Ventilador y Monitoreo Gráfico en Python

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 11, en la que se integra el sensor de temperatura analógico LM35 con el ADC de 12 bits de la Raspberry Pi Pico W para construir un sistema de monitoreo y control térmico. El microcontrolador lee el voltaje del sensor cada segundo, lo convierte a grados Celsius aplicando la relación de 10 mV/°C del LM35, y transmite el valor por el puerto USB-serial. Un LED de alerta se activa automáticamente cuando la temperatura supera los 70 °C, simulando el control de un ventilador mediante un transistor 2N2222A como interruptor de potencia. En la computadora, un script de Python con `matplotlib` grafica la temperatura en tiempo real y registra los datos en un archivo CSV para análisis posterior.

I. INTRODUCCIÓN

I-A Descripción General de la Práctica

Esta práctica integra por primera vez tres dominios: la adquisición de señales analógicas (sensor LM35), el control de actuadores de potencia (ventilador mediante transistor 2N2222A) y la visualización de datos en tiempo real desde una computadora (Python con `matplotlib`). El resultado es un sistema embebido completo que mide, decide y comunica, siendo un prototipo directo de sistemas de enfriamiento inteligente empleados en servidores, electrónica de potencia y equipos industriales.

I-B Sensor de Temperatura LM35

El LM35 es un sensor de temperatura integrado de precisión que genera un voltaje de salida linealmente proporcional a la temperatura en grados Celsius. Su característica principal es una sensibilidad de exactamente 10 mV/°C, lo que significa que a 0 °C la salida es 0 V y a 100 °C la salida es 1 V. Esto simplifica la conversión:

$$T [^{\circ}\text{C}] = \frac{V_{out} [\text{V}]}{0.01 [\text{V}/^{\circ}\text{C}]} = V_{out} \times 100 \quad (1)$$

El LM35 opera con una alimentación de 4 V a 30 V, sin embargo en esta práctica se alimenta directamente

desde el pin de 3.3 V de la Pico W, lo que limita el rango de medición a 0 °C–330 °C, suficiente para la aplicación.

I-C Control de Ventilador con Transistor 2N2222A

Los pines GPIO de la Raspberry Pi Pico W entregan una corriente máxima de 12 mA, insuficiente para alimentar directamente un ventilador de computadora que puede requerir varios cientos de miliamperes. Para aislar eléctricamente el microcontrolador de la carga se utiliza el transistor NPN **2N2222A** como interruptor de colector abierto: cuando el GPIO de control (GP15) pone en alto, la base del transistor se polariza, cerrando el circuito entre colector y emisor y permitiendo que la corriente del ventilador fluya desde su fuente de alimentación independiente hacia GND, sin pasar por la Pico W.

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 sensor LM35, 1 LED de alerta, 1 resistencia de 330 Ω (para LED), 1 resistencia de 1 kΩ (base del transistor), 1 transistor 2N2222A, 1 ventilador de computadora 12V, 1 fuente externa de 12V, 1 diodo de protección 1N4007 (flyback) y 1 protoboard.
- **Software:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, Python 3 con librerías `pyserial` y `matplotlib`.

II-B Conexiones del Circuito

Tabla I: Asignación de pines y conexiones del sistema

Componente	Terminal	Conexión en Pico W
LM35	VCC	Pin 36 (3.3 V)
LM35	GND	GND
LM35	OUT	GP26 (ADC0, pin 31)
LED	Ánodo	GP15 (vía 330 Ω)
LED	Cátodo	GND
2N2222A	Base	GP15 (vía 1 k Ω)
2N2222A	Colector	Terminal – del ventilador
2N2222A	Emisor	GND
Ventilador	+	12V fuente externa

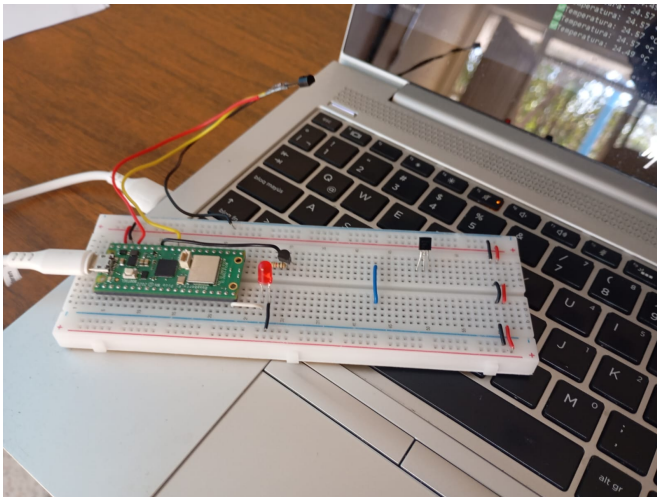


Figura 1: Esquemático del sistema: sensor LM35 en GP26, LED en GP15, transistor 2N2222A controlando el ventilador de 12 V.

II-C Estructura del Proyecto y Configuración de CMake

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)

project(PRACTICA_11)

pico_sdk_init()

add_executable(Sensor
    Sensor.c
)

target_link_libraries(Sensor
    pico_stdlib
```

```
hardware_adc
pico_stdio_usb
)
```

```
pico_enable_stdio_usb(Sensor 1)
pico_enable_stdio_uart(Sensor 0)

pico_add_extra_outputs(Sensor)
```

II-D Implementación del Código en C – Sensor.c

CÓDIGO FUENTE – Sensor.c

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"

#define LED_PIN 15
#define ADC_PIN 26
#define UMBRAL_TEMP 70.0

int main() {
    stdio_init_all();

    // Inicializar LED / control de ventilador
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    // Inicializar ADC
    adc_init();
    adc_gpio_init(ADC_PIN); // GP26 como entrada analógica
    adc_select_input(0);    // Canal ADC0

    while (true) {
        uint16_t raw = adc_read();
        float voltaje = raw * 3.3f / 4095.0f;
        float temperatura = voltaje / 0.01f;
        ; // 10 mV/ C

        printf("Temperatura: %.2f C\n", temperatura);

        if (temperatura >= UMBRAL_TEMP) {
            gpio_put(LED_PIN, 1); // Encender LED / ventilador
            printf("Temperatura ALTA\n");
        } else {
            gpio_put(LED_PIN, 0); // Apagar
        }

        sleep_ms(1000);
    }

    return 0;
}
```

II-D1. Cadena de conversión temperatura

El programa aplica la siguiente cadena de cálculo en cada iteración:

1. `adc_read()` devuelve un entero de 12 bits $N \in [0, 4095]$.
2. Se convierte a voltaje: $V = N \times 3.3/4095$.
3. Se aplica la sensibilidad del LM35: $T = V/0.01 = V \times 100$.
4. Si $T \geq 70^\circ\text{C}$ se activa GP15 (LED + base del transistor $\rightarrow$ ventilador ON).
5. Se transmite el valor por USB-serial y se espera 1s.

II-E Script de Python para Monitoreo Gráfico – grafica.py

SCRIPT PYTHON – grafica.py

```
import re, csv, time
from collections import deque
import serial
import matplotlib.pyplot as plt

PORT = "COM4" # Ajustar al puerto
real
BAUD = 115200
N = 100 # Muestras visibles
CSV_FILE = "temperatura_log.csv"

patron_temp = re.compile(r"Temperatura:\s
*([+]?[0-9]+(?:\.[0-9]+)?)")
ser = serial.Serial(PORT, BAUD, timeout=1)
time.sleep(2)

x_buf = deque(maxlen=N)
y_buf = deque(maxlen=N)

csv_file = open(CSV_FILE, mode="w", newline
=" ", encoding="utf-8")
writer = csv.writer(csv_file)
writer.writerow(["tiempo_s", "temperatura_c
"])

plt.ion()
fig, ax = plt.subplots()
line, = ax.plot([], [])
ax.set_title("Temperatura LM35 en Raspberry
Pi Pico")
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Temperatura (C)")
ax.grid(True)
inicio = time.time()

try:
    while True:
        dato = ser.readline().decode(errors
="ignore").strip()
        if not dato:
```

```
            continue
        m = patron_temp.search(dato)
        if not m:
            continue
        temperatura = float(m.group(1))
        t = time.time() - inicio
        x_buf.append(t)
        y_buf.append(temperatura)
        writer.writerow([f"{t:.2f}", f"{
temperatura:.2f}"])
        csv_file.flush()
        line.set_xdata(list(x_buf))
        line.set_ydata(list(y_buf))
        if len(x_buf) > 1:
            ax.set_xlim(x_buf[0], x_buf
[-1])
            y_min = min(y_buf) - 2
            y_max = max(y_buf) + 2
            if y_min == y_max:
                y_max += 1
            ax.set_ylim(y_min, y_max)
        fig.canvas.draw()
        fig.canvas.flush_events()
        plt.pause(0.01)
except KeyboardInterrupt:
    print("\nCaptura detenida por el
usuario.")
finally:
    csv_file.close()
    ser.close()
    plt.ioff()
    plt.show()
```

El script establece comunicación serial con la Pico W, extrae el valor numérico de temperatura mediante una expresión regular, actualiza en tiempo real la gráfica de temperatura vs. tiempo con matplotlib y guarda cada muestra en el archivo `temperatura_log.csv` para análisis posterior.

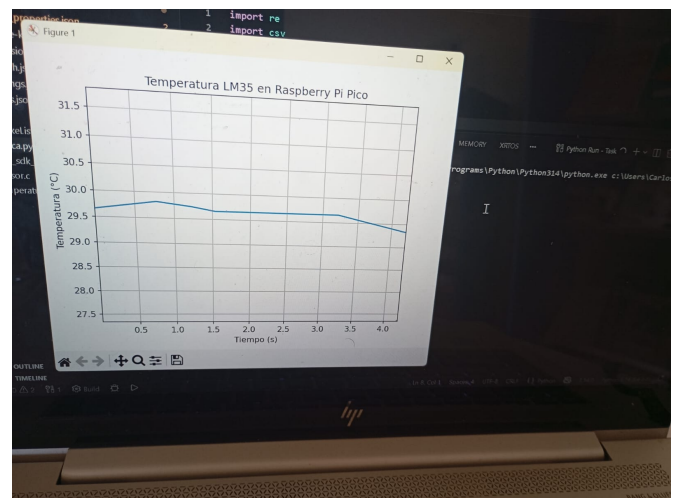


Figura 2: Gráfica de temperatura en tiempo real generada por `grafica.py` con matplotlib.

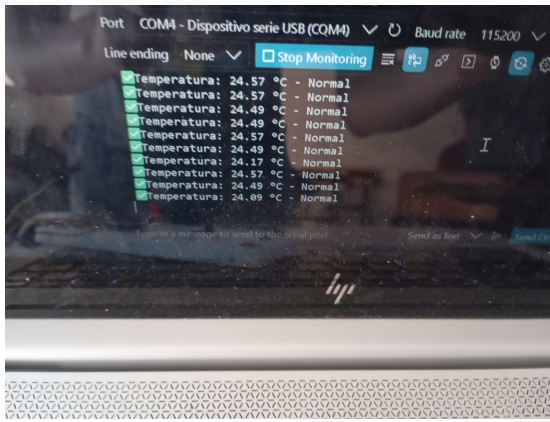


Figura 3: Monitor mostrando la lectura de la temperatura exacta del sensor.

III. PREGUNTAS DE REPASO

III-A ¿Qué es un convertidor analógico-digital (ADC)?

Un convertidor analógico-digital (ADC, por sus siglas en inglés *Analog-to-Digital Converter*) es un circuito electrónico que toma una señal continua (analógica), como un voltaje variable, y la transforma en una secuencia de números enteros (digitales) que representan el valor de la señal en instantes discretos de tiempo. Este proceso implica tres etapas: **muestreo** (tomar medidas periódicas de la señal), **cuantización** (asignar cada muestra al nivel discreto más cercano dentro del rango del conversor) y **codificación** (expresar ese nivel como un número binario). El ADC es el puente esencial entre el mundo físico analógico y el procesamiento digital de un microcontrolador.

III-B ¿Qué resolución tiene el ADC del Raspberry Pi Pico?

La Raspberry Pi Pico W integra un ADC de **12 bits**, lo que significa que puede representar $2^{12} = 4096$ niveles discretos distintos, numerados de 0 a 4095. Con una tensión de referencia de 3.3 V, la resolución (variación mínima de voltaje detectable) es:

$$\text{Resolución} = \frac{3.3 \text{ V}}{4095} \approx 0.806 \text{ mV/nivel} \quad (2)$$

III-C ¿Por qué es necesario convertir el valor digital a voltaje?

El ADC devuelve un número entero proporcional al voltaje de entrada, pero ese número por sí solo no tiene unidades físicas interpretables. Para obtener el voltaje

real se aplica la relación lineal:

$$V = \frac{N \times V_{ref}}{N_{max}} = \frac{N \times 3.3}{4095} \quad (3)$$

Esta conversión es necesaria porque la magnitud física medida (temperatura, en este caso) se obtiene a partir del voltaje, no del número crudo. Sin convertir primero a voltaje, no se podría aplicar la relación física del sensor (10 mV/°C) para obtener la temperatura en grados Celsius.

III-D ¿Cómo se obtiene la temperatura a partir del voltaje del sensor LM35?

El LM35 tiene una sensibilidad de exactamente 10 mV/°C. Por lo tanto, la temperatura se calcula dividiendo el voltaje medido entre ese factor de escala:

$$T [^{\circ}\text{C}] = \frac{V_{out} [\text{V}]}{0.01 [\text{V}/^{\circ}\text{C}]} = V_{out} \times 100 \quad (4)$$

Por ejemplo, si el ADC lee un valor de 1241 (correspondiente a $\approx 1.0 \text{ V}$), la temperatura calculada sería 100 °C. En el código esto se implementa como `temperatura = voltaje / 0.01f`.

III-E ¿Qué sucede si aumenta la resolución del ADC?

Al aumentar la resolución del ADC (por ejemplo de 12 a 16 bits) se incrementa el número de niveles de cuantización de 4096 a 65536, reduciendo el paso mínimo detectable de 0.806 mV a aproximadamente 50 μV . En términos prácticos para este sistema, un ADC de 16 bits permitiría detectar variaciones de temperatura de apenas 0.005 °C en lugar de los 0.08 °C del ADC de 12 bits, mejorando la precisión de la medición. Sin embargo, una mayor resolución también implica mayor tiempo de conversión y mayor sensibilidad al ruido eléctrico, por lo que debe equilibrarse con los requisitos reales de la aplicación.

III-F ¿Por qué es útil representar los datos mediante una gráfica?

Una gráfica de temperatura en función del tiempo permite identificar patrones, tendencias y anomalías que serían imposibles de detectar observando valores numéricos aislados. Entre sus ventajas destacan la detección visual de calentamiento gradual o enfriamiento rápido, la identificación de ciclos periódicos, la correlación entre eventos (como el encendido del ventilador y la bajada de temperatura), y la comunicación intuitiva de resultados.

En ingeniería de sistemas embebidos, la visualización en tiempo real es una herramienta de depuración y validación indispensable.

III-G ¿Qué aplicaciones industriales tiene la medición de temperatura con microcontroladores?

La medición de temperatura con microcontroladores es omnipresente en la industria moderna. Algunas aplicaciones relevantes son el control de temperatura en hornos y reactores químicos, la supervisión térmica de motores eléctricos y variadores de frecuencia, los sistemas de gestión térmica de baterías en vehículos eléctricos (BMS), la refrigeración activa de servidores y centros de datos, el control de procesos en líneas de manufactura de alimentos y farmacéuticos, y los sistemas de alarma contra incendios en instalaciones industriales. En todos estos casos, la capacidad de medir, comparar con un umbral y actuar un actuador (ventilador, resistencia, válvula) en tiempo real es la base del control automático.

III-H ¿Qué ventajas tiene registrar datos en el tiempo en lugar de observar solo un valor instantáneo?

El registro temporal (como el archivo CSV generado por `grafica.py`) ofrece ventajas que van más allá de la observación instantánea:

- **Análisis post-proceso:** permite aplicar filtros, promedios o algoritmos de detección de anomalías sobre el historial completo.
- **Trazabilidad:** en aplicaciones industriales y médicas, el registro con marca de tiempo es un requisito legal para auditorías.
- **Detección de tendencias:** un valor instantáneo de 65°C puede ser normal o alarmante dependiendo de si viene subiendo desde 20°C en 10 minutos o lleva horas estable; solo el historial lo revela.
- **Calibración y validación:** comparar registros históricos con valores de referencia permite detectar derivas del sensor o del sistema.

IV. CONCLUSIONES

- La integración del sensor LM35 con el ADC de 12 bits de la Pico W demostró ser directa y precisa gracias a la linealidad del sensor: la cadena de conversión (valor ADC → voltaje → temperatura) se implementa en dos líneas de código sin necesidad de tablas de calibración ni compensación no lineal.

- El uso del transistor 2N2222A como interruptor de potencia es una solución fundamental en sistemas embebidos: protege al microcontrolador de corrientes excesivas y permite controlar cargas de mayor potencia (ventiladores, relés, solenoides) con una señal lógica de 3.3 V y microamperes.
- El script de Python con `matplotlib` elevó significativamente el valor del sistema al agregar visualización en tiempo real y registro persistente en CSV, mostrando cómo la combinación de un microcontrolador (adquisición de datos) con una computadora (procesamiento y visualización) es un paradigma poderoso en sistemas de instrumentación modernos.
- La librería `pico_stdio_usb` en el `CMakeLists.txt` y las directivas `pico_enable_stdio_usb` / `pico_enable_stdio_uart` son esenciales para establecer la comunicación serial por USB que habilita tanto el monitor serial como el script de Python.
- Como mejora futura se podría implementar control PWM sobre el ventilador para regular su velocidad de forma proporcional a la temperatura, en lugar de un control todo-o-nada, mejorando la eficiencia energética y reduciendo el ruido acústico del sistema.

ANEXOS

A. Código fuente — Sensor.c

ANEXO A — Sensor.c (completo)

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"

#define LED_PIN 15
#define ADC_PIN 26
#define UMBRAL_TEMP 70.0

int main() {
    stdio_init_all();
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);
    adc_init();
    adc_gpio_init(ADC_PIN);
    adc_select_input(0);

    while (true) {
        uint16_t raw = adc_read();
        float voltaje = raw * 3.3f / 4095.0f;
        float temperatura = voltaje / 0.01f
```

```

;

printf("Temperatura: %.2f C\n",
      temperatura);

if (temperatura >= UMBRAL_TEMP) {
    gpio_put(LED_PIN, 1);
    printf("Temperatura ALTA\n");
} else {
    gpio_put(LED_PIN, 0);
}
sleep_ms(1000);
}
return 0;
}

```

```

set_ydata(list(y_buf))
if len(x_buf) > 1:
    ax.set_xlim(x_buf[0], x_buf[-1])
    ax.set_ylim(min(y_buf)-2, max(y_buf)+2)
    fig.canvas.draw(); fig.canvas.flush_events(); plt.pause(0.01)
except KeyboardInterrupt:
    print("Captura detenida.")
finally:
    csv_file.close(); ser.close(); plt.ioff(); plt.show()

```

B. Script Python – grafica.py

ANEXO B – grafica.py (completo)

```

import re, csv, time
from collections import deque
import serial
import matplotlib.pyplot as plt

PORT = "COM4"; BAUD = 115200; N = 100
CSV_FILE = "temperatura_log.csv"
patron_temp = re.compile(r"Temperatura:\s*([+-]?\d+(:\.\d+)?)")
ser = serial.Serial(PORT, BAUD, timeout=1)
time.sleep(2)
x_buf = deque(maxlen=N); y_buf = deque(maxlen=N)
csv_file = open(CSV_FILE, "w", newline="", encoding="utf-8")
writer = csv.writer(csv_file)
writer.writerow(["tiempo_s", "temperatura_c"])
plt.ion(); fig, ax = plt.subplots(); line, = ax.plot([], [])
ax.set_title("Temperatura LM35"); ax.set_xlabel("Tiempo (s)"); ax.set_ylabel("Temperatura (C)"); ax.grid(True)
inicio = time.time()
try:
    while True:
        dato = ser.readline().decode(errors="ignore").strip()
        m = patron_temp.search(dato)
        if not m: continue
        temperatura = float(m.group(1))
        t = time.time() - inicio
        x_buf.append(t); y_buf.append(temperatura)
        writer.writerow([f"{t:.2f}", f"{temperatura:.2f}"])
        csv_file.flush()
        line.set_xdata(list(x_buf)); line.

```

C. Archivo CMakeLists.txt

ANEXO C – CMakeLists.txt (completo)

```

cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)

project(PRACTICA_11)

pico_sdk_init()

add_executable(Sensor
    Sensor.c
)

target_link_libraries(Sensor
    pico_stdlib
    hardware_adc
    pico_stdio_usb
)

pico_enable_stdio_usb(Sensor 1)
pico_enable_stdio_uart(Sensor 0)

pico_add_extra_outputs(Sensor)

```

D. Diagrama en Protoboard

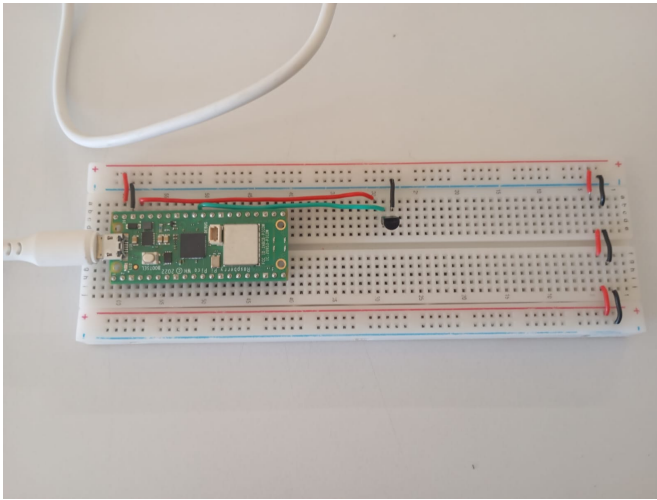


Figura 4: Diagrama en protoboard: LM35 en GP26, LED/-transistor 2N2222A en GP15 y ventilador con fuente de 12 V externa.